

```
In [12]: import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
import numpy as np
import matplotlib.pyplot as plt # También asegúrate de tener este para las g
from sklearn.metrics import confusion_matrix, accuracy_score, roc_auc_score,

# Carga limpia del archivo
df = pd.read_csv('Datos.csv')
```

Introducción

En este proyecto, se pretenden analizar distintos datos obtenidos sobre la obesidad en 3 países diferentes, México, Perú y Colombia. Esta base de datos fue obtenida desde la plataforma Kaggle, los datos presentes en esta fueron obtenidos de un estudio realizado en 2019 por Fabio Mendoza y Alexis De la Hoz. Este estudio se publicó en la revista científica Data In Brief, lo que aporta seguridad y fiabilidad sobre los datos. A parte, el estudio contiene un total de 110 citaciones a fuentes/estudios y 9 referencias, lo que soporta tanto los datos como la clasificación de estos, las variables utilizadas, su significado y magnitudes. Este dataset contiene un total de 2111 registros y 17 columnas.

La problemática escogida es la obesidad, esto debido a su alto porcentaje en la población mexicana. Según Global Obesity Observatory, México se encuentra en el puesto 31 en el ranking de países con obesidad, con un porcentaje de 36.86%. Según la Encuesta Nacional de Salud y Nutrición, realizada desde 2020 a 2023, se obtuvo que la prevalencia nacional de obesidad fue de un 37.1%. En mujeres, el porcentaje de prevalencia de obesidad fue 41%, mientras que en hombres fue un 33%. Estos datos fueron presentados en un estudio, donde también aportaban información significativa sobre la obesidad en México, como la edad de los entrevistados y clasificación por edades teniendo en cuenta su porcentaje de prevalencia de grasa.

Mediante los modelos que se van a realizar en esta investigación se pretende clasificar a las personas en dos categorías: TieneSobrepeso y NoTieneSobrepeso. El enfoque de clasificación es adecuado porque permite transformar el problema en una herramienta de diagnóstico preventivo. En lugar de predecir un valor continuo, buscamos asignar a un individuo a una categoría basada en sus hábitos, lo que facilita la toma de decisiones en salud pública. Con estos modelos realizaremos una clasificación más general, y estará puramente basada en hábitos personales, incluyendo también la edad ya que esta juega un factor importante en esta problemática. Para esto, se binarizó la variable de salida del dataset, agrupando las clases Overweight (nivel I y II) y Obesity (tipo I, II y III) como TieneSobrepeso que corresponde a la salida (1), y las clases Insufficient Weight y Normal Weight como NoTieneSobrepeso que corresponde a la salida (0).

Se eliminaron las variables de altura y peso del conjunto de características para evitar fuga de datos, ya que el nivel de obesidad se calcula directamente a partir de estas,

basado en la fórmula del Índice de Masa Corporal (IMC), lo que haría que el modelo simplemente replicara ese cálculo en lugar de aprender patrones reales basados en los hábitos de las personas. De esta forma, el enfoque del proyecto se centra en entender qué hábitos de vida y características personales permiten predecir si una persona tiene o no sobrepeso, utilizando y comparando múltiples algoritmos de clasificación estudiados durante el curso.

Exploración y comprensión del conjunto de datos

Para poder entender los datos y los diferentes modelos de clasificación que se realizarán, primero se deben entender cada una de las variables presentes en el dataset:

Variable	Descripción	Tipo	Relación con la variable de salida
Gender	Género de la persona	Categórica	Posible diferencia según datos de la ESANUT
Age	Edad en años	Numérica	Factor de riesgo debido a problemas relacionados con el cambio de metabolismo y acumulación de hábitos a largo plazo
family_history	Antecedentes de familiares con sobrepeso	Categórica	Predictor importante debido a la predisposición de genética y sesgo de hábitos debido al entorno familiar
FAVC	Consumo frecuente de alimentos hipercalóricos	Categórica	Relación directa con el balance energético y ganancia de grasa
FCVC	Frecuencia de consumo de vegetales (1-3)	Numérica	Factor con influencia inversamente proporcional, donde se espera que a mayor consumo, haya menor riesgo de sobrepeso
NCP	Número de comidas principales al día	Numérica (Ordinal)	Indica la cantidad de ingestas grandes en el día, que tiene influencia en el control de la saciedad
CAEC	Consumo de alimentos entre comidas	Categórica	Representa la ingesta de calorías extra, suele estar ligada a snacks o alimentos procesados por lo que se estima una relación directamente proporcional con la variable de salida
SMOKE	Hábito de fumar	Categórica	Variable que puede influir en el metabolismo o sustituir comportamientos alimentarios
CH20	Consumo diario de agua (litros)	Numérica	Puede relacionarse con la variable de salida ya que suele indicar un mejor metabolismo

Variable	Descripción	Tipo	Relación con la variable de salida
SCC	Monitoreo diario de calorías	Categórica	Indica cuidado de la salud y alimentos que se consumen, puede ser un factor preventivo en una dieta, gimnasio...
FAF	Frecuencia de actividad física	Numérica	Gasto energético directo, se estima que es el principal factor protectos contra la obesidad
TUE	Tiempo de uso de dispositivos	Numérica	Indicador indirecto del sedentarismo de una persona
CALC	Frecuencia de consumo de alcohol	Categórica	EL alcohol aporta calorías vacías que impacta directamente con el aumento de peso
MTRANS	Medio de transporte habitual	Categórica	Refleja el nivel de actividad física no programada, como caminar en vez de usar el coche
Obesity	Nivel de obesidad (variable objetivo)	Categórica	Variable a predecir, binarizada en: 1 (TieneSobrepeso) y 0 (NoTieneSobrepeso)

Al observar nuestros datos y como se miden, resaltan todos los datos de tipo numérico, pues no hay ninguno que sea entero, todos contienen decimales. Esto es debido a una técnica llamada SMOTE (Synthetic Minority Over-sampling Technique) que los dos investigadores implementaron. Esta técnica consiste en equilibrar el conjunto de datos mediante la creación de nuevas muestras artificiales para los grupos que contienen menos datos. En vez de duplicar registros, SMOTE selecciona vecinos cercanos con respecto a las características de los registros y traza líneas entre ellos para generar nuevos puntos sintéticos, finalmente los registros se obtienen de dichos puntos. La técnica SMOTE fue utilizada para generar un total de 1613 registros, que representaron el 77% del dataset. Los otros 498 registros fueron obtenidos mediante una encuesta web. Aunque solo el 23% de los datos hayan sido registros reales, SMOTE es una técnica validada en la ciencia de datos. La implementación de esta técnica permite que los modelos de clasificación que vamos a crear tengan una cantidad de muestra suficiente para cada categoría original.

Se clasificarán las personas en dos clases mediante el nivel de obesidad; en el estudio original se dividen como "Insufficient Weight", "Normal Weight", "Overweight I y II" y "Obesity type I, II, III". Se determinará que una persona tiene sobrepeso si tiene tanto Overweight de cualquier tipo como Obesity de cualquier tipo. Esto hace que el balance de clases esté un poco desproporcionado al contar con más casos que se asignarán a la clase TieneSobrepeso. Esta clase se contará como una salida positiva (1) y contiene el 73% de los registros del dataset, siendo el 27% restante los casos que se consideran como NoTieneSobrepeso. Se estará manteniendo este balance de clases en las fases de train y test para que el modelo aprenda realmente sobre qué hábitos influyen en el sobrepeso y no se confunda a la hora de tener diferentes distribuciones. Esta

preservación del balance es fundamental para realizar una evaluación justa del modelo, asegurando que las métricas de desempeño reflejen su capacidad real de generalización ante la proporción real de las clases. Debido a esto, la evaluación del modelo no se basará únicamente en el Accuracy, ya que este podría estar sesgado hacia la clase mayoritaria. Se dará prioridad al F1-Score y la Matriz de Confusión para garantizar que el modelo identifica correctamente a ambos grupos. Con respecto al train y test, los datos se dividirán con un 80% de train y un 20% de test para observar la eficacia de los modelos.

```
In [3]: import matplotlib.pyplot as plt

# Binarizar variable de salida
obeso_labels = [
    'Obesity_Type_I', 'Obesity_Type_II', 'Obesity_Type_III',
    'Overweight_Level_I', 'Overweight_Level_II'
]
df['TieneSobrepeso'] = df['Obesity'].apply(lambda x: 1 if x in obeso_labels
                                           else 0)

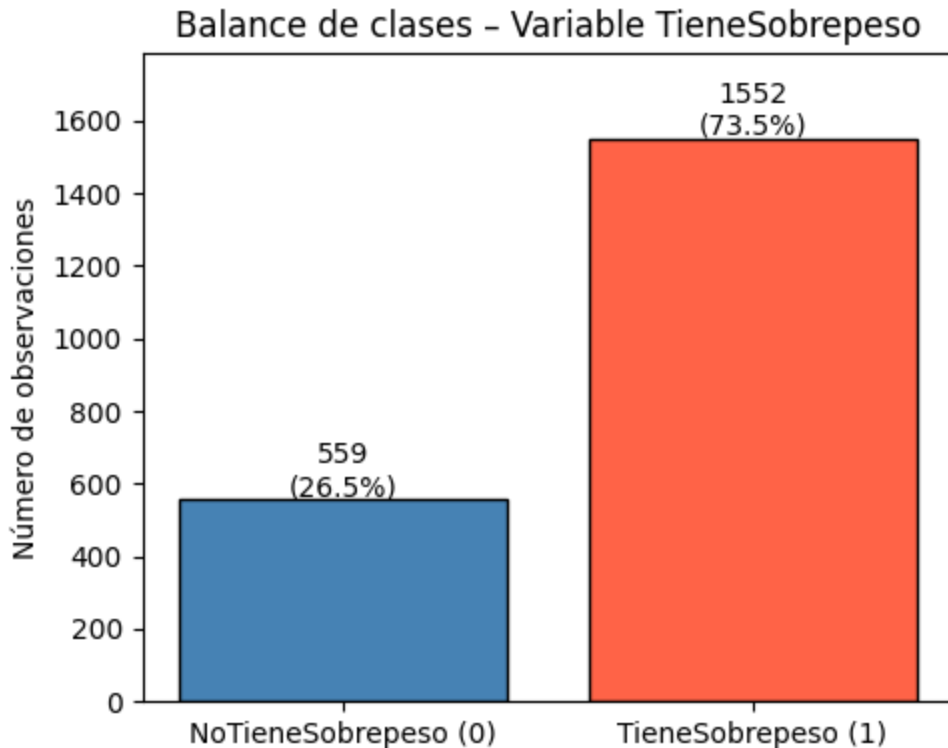
# Balance de clases
conteo = df['TieneSobrepeso'].value_counts()
proporcion = df['TieneSobrepeso'].value_counts(normalize=True) * 100
balance = pd.DataFrame({
    'Clase': ['NoTieneSobrepeso (0)', 'TieneSobrepeso (1)'],
    'Conteo': [conteo[0], conteo[1]],
    'Porcentaje': [proporcion[0], proporcion[1]]
})

print('Balance de clases en la variable de salida:')
print(balance.to_string(index=False))

fig, ax = plt.subplots(figsize=(5, 4))
ax.bar(['NoTieneSobrepeso (0)', 'TieneSobrepeso (1)'], [conteo[0], conteo[1]],
       color=['steelblue', 'tomato'], edgecolor='black')
ax.set_ylabel('Número de observaciones')
ax.set_title('Balance de clases – Variable TieneSobrepeso')
for i, v in enumerate([conteo[0], conteo[1]]):
    ax.text(i, v + 10, f'{v}\n({proporcion[i]:.1f}%)', ha='center', fontsize=12)
ax.set_ylim(0, max(conteo[0], conteo[1]) * 1.15)
plt.tight_layout()
plt.show()
```

Balance de clases en la variable de salida:

	Clase	Conteo	Porcentaje
NoTieneSobrepeso (0)		559	26.480341
TieneSobrepeso (1)		1552	73.519659



Preparación y tratamiento de los datos

La preparación de datos sigue el mismo proceso realizado en el proyecto del primer parcial y las actividades del segundo parcial, donde se trabajó con este mismo dataset. A continuación se resumen las decisiones tomadas y se incluye una corrección en la etapa de estandarización que se identificó como mejora respecto al proyecto anterior. Para esto nos vamos a fijar en 3 aspectos clave:

El primer aspecto son los diferentes tipos de variables. Podemos observar que nuestra base de datos contiene tanto variables numéricas como de tipo string. Para nuestros modelos, necesitamos que todas las variables sean numéricas. Por eso, estaremos utilizando 2 técnicas de codificación:

Ordinal Encoding: esta técnica consiste en convertir datos categóricos a datos numéricos donde se le asigna un entero único a cada categoría respetando su orden lógico. Se estará utilizando en variables como CAEC y CALC, donde existe un orden de frecuencia (no < Sometimes < Frequently < Always), y en MTRANS, donde se establece una jerarquía basada en el nivel de sedentarismo del transporte (Walking < Bike < Motorbike < Public_Transportation < Automobile). **Label Encoding:** esta técnica consiste en cambiar los valores de tipo string a valores numéricos binarios 0 y 1, indicando presencia o ausencia del atributo. Será utilizada en variables con solo 2 respuestas posibles: Gender, family_history, FAVC, SMOKE y SCC.

El segundo aspecto es el riesgo de fuga de datos (Data Leakage). La variable de salida original del dataset (Obesity) indica el nivel de obesidad de cada persona, el cual se

determina directamente a partir del peso y la estatura mediante el cálculo del IMC. Si se mantuvieran estas variables como características de entrada, los modelos no aprenderían patrones basados en los hábitos de las personas, sino que simplemente replicarían el cálculo del IMC, anulando por completo el propósito del análisis. Por ello, se decide eliminar Height y Weight del conjunto de características. Aunque esta decisión puede resultar en métricas de desempeño numéricamente menores, es la única vía para obtener modelos útiles que permitan entender qué hábitos influyen realmente en el sobrepeso.

El tercer aspecto es la estandarización y su correcta aplicación. Para los modelos que lo requieren, como SVM y la red neuronal, se aplica StandardScaler para centrar los datos en una media de 0 y una desviación estándar de 1, evitando que variables con rangos numéricos más amplios (como la edad) dominen sobre variables binarias. Es fundamental que el escalamiento se ajuste únicamente con los datos de entrenamiento y luego se aplique a los datos de prueba, sin incluirlos en el cálculo. De lo contrario, se estaría filtrando información del conjunto de prueba hacia el proceso de entrenamiento, generando fuga de datos en la etapa de preprocesamiento. Finalmente, los datos se dividen con un 80% para entrenamiento y un 20% para prueba, utilizando estratificación para mantener el balance de clases original (73.5% TieneSobrepeso / 26.5% NoTieneSobrepeso) en ambos conjuntos. La razón del uso de estratificación es debido a desbalance de clases tan amplio que tiene nuestro dataset. Si no aplicamos esta técnica, los modelos que vamos a crear se van a entrenar con una proporción distinta a la realidad, con tantos registros clasificados como "TieneSobrepeso", el modelo podría decir que todos los registros tienen sobrepeso y estaría acertando el 73% de las veces con nuestros datos, pero con otros datos estaría fallando y perdiendo todo el propósito que es que aprenda a clasificar correctamente.

```
In [4]: # =====
# Preparación y tratamiento de los datos
# =====

# 1. Eliminar Height, Weight y Obesity (fuga de datos)
df = df.drop(columns=['Height', 'Weight', 'Obesity'])
print(f"Columnas restantes: {list(df.columns)}")
print(f"Shape: {df.shape}")

# 2. Ordinal Encoding: variables con orden lógico
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder

caec_order = ['no', 'Sometimes', 'Frequently', 'Always']
calc_order = ['no', 'Sometimes', 'Frequently', 'Always']
mtrans_order = ['Walking', 'Bike', 'Motorbike', 'Public_Transportation', 'Au

oe = OrdinalEncoder(
    categories=[caec_order, calc_order, mtrans_order],
    handle_unknown='use_encoded_value', unknown_value=-1
)
df[['CAEC', 'CALC', 'MTRANS']] = oe.fit_transform(df[['CAEC', 'CALC', 'MTRANS']])
```

```

# 3. Label Encoding: variables binarias (yes/no, Female/Male)
le = LabelEncoder()
for col in ['Gender', 'family_history', 'FAVC', 'SMOKE', 'SCC']:
    df[col] = le.fit_transform(df[col])

# 4. Separar features y variable objetivo
feature_cols = [c for c in df.columns if c != 'TieneSobrepeso']
X = df[feature_cols]
y = df['TieneSobrepeso']

# 5. Partición Train/Test (80/20, estratificado para mantener balance)
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 6. Verificar balance de clases en los 3 grupos
print("\n--- Balance de clases ---")
print(f"Original : {y.mean()*100:.1f}% TieneSobrepeso | {(1-y.mean())*100:.1f}% NoTieneSobrepeso")
print(f"Train      : {y_train.mean()*100:.1f}% TieneSobrepeso | {(1-y_train.mean())*100:.1f}% NoTieneSobrepeso")
print(f"Test       : {y_test.mean()*100:.1f}% TieneSobrepeso | {(1-y_test.mean())*100:.1f}% NoTieneSobrepeso")
print(f"\nTrain: {len(X_train)} registros | Test: {len(X_test)} registros")

# 7. Estandarización SOLO para modelos que lo requieren (SVM, Red Neuronal)
# fit() únicamente con train para evitar fuga de datos
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train) # fit + transform en train
X_test_sc = scaler.transform(X_test)      # solo transform en test

print(f"\nDatos escalados listos. Shape train: {X_train_sc.shape} | test: {X_test_sc.shape}")

```

Columnas restantes: ['Gender', 'Age', 'family_history', 'FAVC', 'FCVC', 'NCAP', 'CAEC', 'SMOKE', 'CH20', 'SCC', 'FAF', 'TUE', 'CALC', 'MTRANS', 'TieneSobrepeso']

Shape: (2111, 15)

--- Balance de clases ---

Original : 73.5% TieneSobrepeso | 26.5% NoTieneSobrepeso

Train : 73.5% TieneSobrepeso | 26.5% NoTieneSobrepeso

Test : 73.5% TieneSobrepeso | 26.5% NoTieneSobrepeso

Train: 1688 registros | Test: 423 registros

Datos escalados listos. Shape train: (1688, 14) | test: (423, 14)

Regresión Logística

Se realizó validación cruzada con la técnica K-Fold (k=10) sobre los datos de entrenamiento, obteniendo una exactitud promedio de 86.55% con una desviación estándar de 1.87%. La variabilidad entre folds fue baja (de 83.43% a 90.48%), lo que indica que el modelo es consistente y no depende de una partición específica de los

datos. Se entrenó el modelo de regresión logística con la totalidad del conjunto de entrenamiento y se evaluó sobre el conjunto de prueba bajo tres umbrales de decisión distintos. Con un umbral de 0.35, el modelo alcanzó una exactitud de 88.18%, una sensibilidad de 96.78% y una especificidad de 64.29%. Este umbral es el más agresivo en la detección de casos positivos: al bajar el punto de corte, el modelo clasifica como TieneSobrepeso a más personas, lo que reduce los falsos negativos pero incrementa los falsos positivos, reflejándose en la especificidad más baja de los tres escenarios. En un contexto de salud pública, este umbral es el más recomendable, ya que es preferible identificar a todas las personas en riesgo aunque algunas clasificaciones sean incorrectas.

Con un umbral de 0.50, el modelo obtuvo su mayor exactitud con 88.89%, una sensibilidad de 94.53% y una especificidad de 73.21%. Este umbral representa el balance estándar entre ambas métricas y ofrece el mejor desempeño general en términos de exactitud. Con un umbral de 0.65, la exactitud fue de 87.47%, la sensibilidad bajó a 90.68% y la especificidad subió a 78.57%. Al elevar el umbral, el modelo se vuelve más conservador: solo clasifica como TieneSobrepeso a quienes tienen una probabilidad alta, lo que reduce los falsos positivos pero deja pasar más casos reales de sobrepeso sin detectar.

La curva ROC obtenida muestra un AUC de 0.9062, lo que indica que el modelo tiene una buena capacidad discriminativa. Comparando con los resultados de la validación cruzada, que reportó una exactitud promedio de 86.55%, el modelo evaluado en el conjunto de prueba con umbral 0.50 alcanzó un 88.89%, ligeramente superior. Esta diferencia es pequeña y esperada: la validación cruzada estima el desempeño promedio sobre múltiples particiones, por lo que su resultado es más conservador y representativo del comportamiento general del modelo. La consistencia entre ambos valores confirma que el modelo generaliza bien y no presenta sobreajuste. En cuanto a los coeficientes del modelo, la variable con mayor peso positivo es la edad (1.177), seguida de family_history (1.013), lo que indica que una mayor edad y tener antecedentes familiares de sobrepeso incrementan significativamente la probabilidad de pertenecer a la clase TieneSobrepeso. Por otro lado, las variables con mayor peso negativo son CAEC (-0.963) y NCP (-0.327), sugiriendo que el consumo entre comidas y el número de comidas principales se asocian con la clase NoTieneSobrepeso en este modelo.

```
In [5]: from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             ConfusionMatrixDisplay, roc_curve, roc_auc_score)

# 1. Validación cruzada (10-fold) sobre datos de entrenamiento
modelo_lr = LogisticRegression(max_iter=1000, random_state=42)
skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
scores = cross_val_score(modelo_lr, X_train_sc, y_train, cv=skf, scoring='accuracy')

print('Exactitud por fold:')
```



```

for i, s in enumerate(scores, 1):
    print(f'  Fold {i:2d}: {s*100:.2f}%')
print(f'\nExactitud promedio: {scores.mean()*100:.2f}%')
print(f'Desviación estándar: {scores.std()*100:.2f}%')

# 2. Entrenar modelo final con todo el conjunto de entrenamiento
modelo_lr.fit(X_train_sc, y_train)

# 3. Probabilidades y evaluación con 3 umbrales
y_prob_lr = modelo_lr.predict_proba(X_test_sc)[: , 1]
umbrales = [0.35, 0.50, 0.65]

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle('Matrices de Confusión por Umbral – Regresión Logística', fontst

print(f"\n{'Umbral':>8} {'Exactitud':>10} {'Sensibilidad':>13} {'Especificidad':>13}")
print('-' * 50)

for ax, umbral in zip(axes, umbrales):
    y_pred = (y_prob_lr >= umbral).astype(int)
    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()

    exactitud = accuracy_score(y_test, y_pred)
    sensibilidad = tp / (tp + fn)
    especificidad = tn / (tn + fp)

    print(f"{'umbral':>8.2f} {'exactitud*100':>9.2f}% {'sensibilidad*100':>12.2f}% {'especificidad*100':>12.2f}%")

    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['NoSc', 'Sc'])
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(f'Umbral = {umbral}\nExact={exactitud*100:.1f}% Sens={sensibilidad*100:.1f}% Espec={especificidad*100:.1f}%')

plt.tight_layout()
plt.show()

# 4. Métricas finales con umbral 0.5 (para tabla comparativa)
y_pred_lr = modelo_lr.predict(X_test_sc)
cm_lr = confusion_matrix(y_test, y_pred_lr)
tn, fp, fn, tp = cm_lr.ravel()

acc_lr = accuracy_score(y_test, y_pred_lr)
sens_lr = tp / (tp + fn)
spec_lr = tn / (tn + fp)
auc_lr = roc_auc_score(y_test, y_prob_lr)

print(f"\n{'='*50}")
print(f" MÉTRICAS – Regresión Logística (umbral 0.5)")
print(f"{'='*50}")
print(f"  Accuracy      : {acc_lr*100:.2f}%")
print(f"  Sensibilidad   : {sens_lr*100:.2f}%")
print(f"  Especificidad  : {spec_lr*100:.2f}%")
print(f"  AUC-ROC       : {auc_lr:.4f}")

# 5. Curva ROC
fpr, tpr, thresholds = roc_curve(y_test, y_prob_lr)

```

```

fig, ax = plt.subplots(figsize=(7, 6))
ax.plot(fpr, tpr, color='steelblue', lw=2, label=f'Reg. Logística (AUC = {auc})')
ax.plot([0, 1], [0, 1], color='gray', linestyle='--', lw=1, label='Clasificación aleatoria')
ax.set_xlabel('Tasa de Falsos Positivos (1 - Especificidad)')
ax.set_ylabel('Tasa de Verdaderos Positivos (Sensibilidad)')
ax.set_title('Curva ROC - Regresión Logística')
ax.legend(loc='lower right')
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# 6. Coeficientes del modelo
coef_lr = pd.DataFrame({
    'Variable': feature_cols,
    'Coeficiente': modelo_lr.coef_[0]
}).sort_values('Coeficiente', ascending=False)

print("\nCoeficientes del modelo (ordenados por peso):")
print(coef_lr.to_string(index=False))

```

Exactitud por fold:

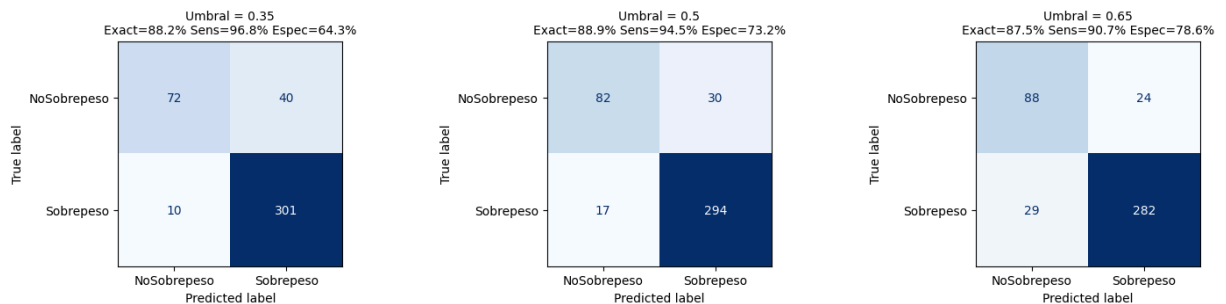
Fold 1: 86.39%
 Fold 2: 83.43%
 Fold 3: 86.39%
 Fold 4: 86.98%
 Fold 5: 85.80%
 Fold 6: 86.98%
 Fold 7: 87.57%
 Fold 8: 87.57%
 Fold 9: 83.93%
 Fold 10: 90.48%

Exactitud promedio: 86.55%

Desviación estándar: 1.87%

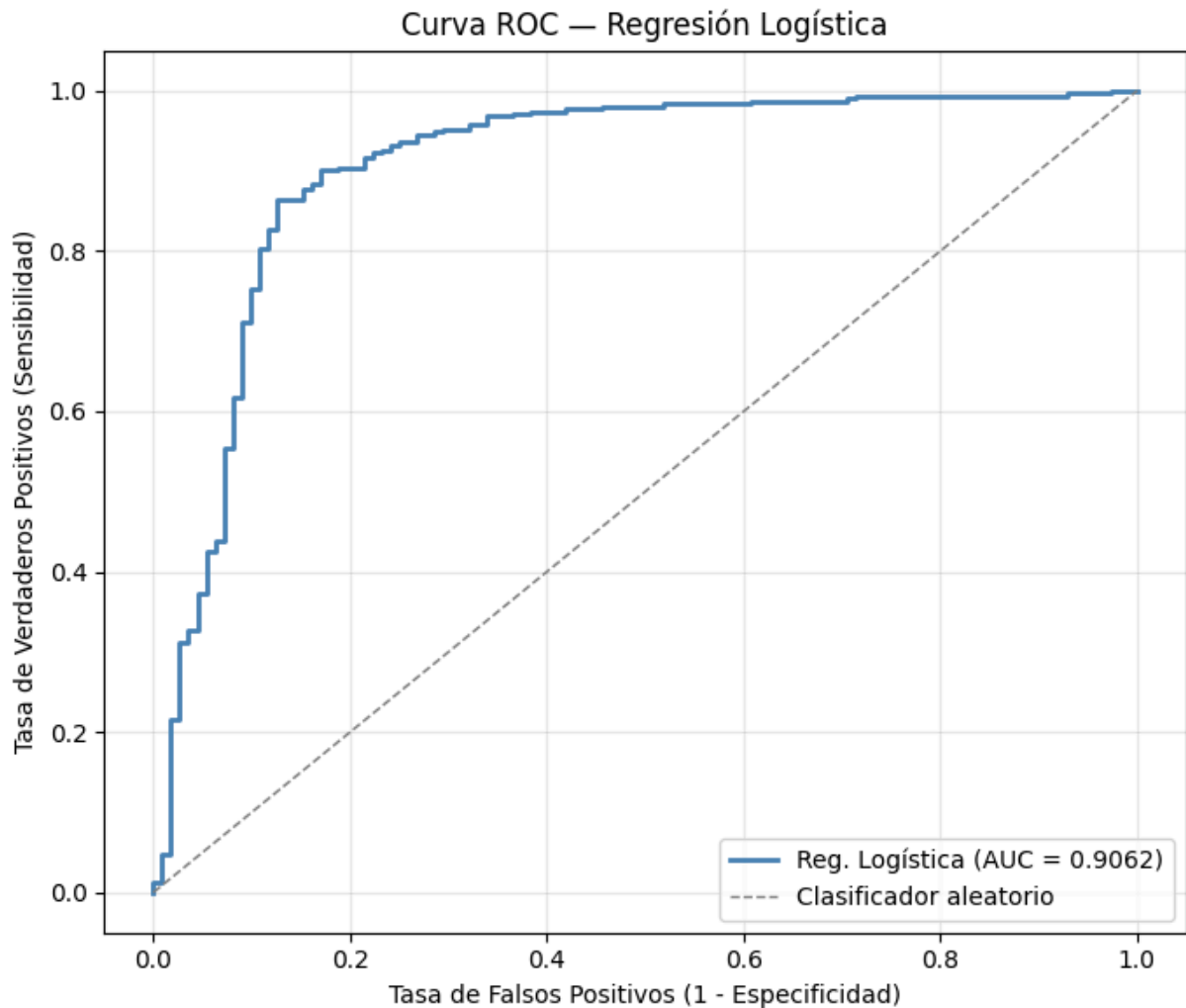
Umbral	Exactitud	Sensibilidad	Especificidad
0.35	88.18%	96.78%	64.29%
0.50	88.89%	94.53%	73.21%
0.65	87.47%	90.68%	78.57%

Matrices de Confusión por Umbral — Regresión Logística



MÉTRICAS – Regresión Logística (umbral 0.5)

Accuracy : 88.89%
 Sensibilidad : 94.53%
 Especificidad : 73.21%
 AUC-ROC : 0.9062



Coefficientes del modelo (ordenados por peso):

Variable	Coefficiente
Age	1.177428
family_history	1.012690
CALC	0.313076
FAVC	0.176165
CH20	0.153866
SCC	0.123863
Gender	0.057291
MTRANS	0.028385
FCVC	0.028173
SMOKE	-0.114421
TUE	-0.118119
FAF	-0.311336
NCP	-0.327252
CAEC	-0.962706

Modelo LDA

LDA busca la proyección que maximiza la separación entre clases respecto a la varianza dentro de cada clase. Al tener 2 clases, se obtiene una única función discriminante (LD1), cuyos coeficientes indican el peso de cada variable en la separación. Obtenemos los coeficientes ordenados por magnitud que nos indican qué variables tienen mayor peso (positiva o negativamente) a la hora de clasificar el sobrepeso. La variable más influyente es "family_history" (+1.957), lo que al igual que en el modelo de regresión logística, puede significar que la influencia familiar y los hábitos de las personas que rodean a alguien en su familia influyen en su nivel de peso. Otra variable con un peso bastante importante fue "CAEC" (-1.360), que obtiene información sobre si la persona consume snacks entre comidas; su signo negativo indica que un mayor consumo entre comidas se asocia curiosamente a la clase NoTieneSobrepeso, lo cual podría explicarse porque esta variable también captura patrones de personas con peso insuficiente o normal que comen con más frecuencia pero en menor cantidad. Las variables SMOKE, CALC y FAVC son las siguientes con un coeficiente suficientemente relevante, pero muy alejadas de las dos principales.

Con respecto a las métricas, el modelo obtiene un Accuracy de 88.65%, lo que indica que clasifica correctamente casi 9 de cada 10 casos. La Sensibilidad de 95.18% es especialmente relevante en este contexto: el modelo detecta correctamente el 95% de las personas que realmente tienen sobrepeso, lo cual es deseable en un problema de salud donde es más costoso no identificar un caso positivo. Sin embargo, la Especificidad de 70.54% es la métrica más baja del modelo, lo que indica que tiene dificultades para identificar correctamente a las personas sin sobrepeso. Esto se debe a que la frontera lineal de LDA no es suficiente para separar perfectamente ambas clases, existiendo una zona de traslape donde el modelo no logra clasificar con confianza. Finalmente, el AUC-ROC de 0.9037 confirma que el modelo tiene un buen desempeño global considerando todos los umbrales posibles.

Comparando con la regresión logística (Accuracy 88.89%, AUC 0.9062), LDA obtiene resultados muy similares, lo cual es esperado ya que ambos son modelos lineales. La principal diferencia es que LDA tiene una sensibilidad ligeramente más alta (95.18% vs 94.53%) pero a costa de una especificidad más baja (70.54% vs 73.21%). Aunque el LDA asume normalidad en las variables, su desempeño es consistente con la Regresión Logística, lo que sugiere que la separación lineal es efectiva para este conjunto de datos.

```
In [6]: # =====
# Linear Discriminant Analysis (LDA)
# =====
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# 1. Ajuste del modelo
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)

# 2. Predicciones y probabilidades
```

```

y_pred_lda = lda.predict(X_test)
y_prob_lda = lda.predict_proba(X_test)[:, 1]

# 3. Métricas
cm_lda = confusion_matrix(y_test, y_pred_lda)
tn, fp, fn, tp = cm_lda.ravel()

acc_lda = accuracy_score(y_test, y_pred_lda)
sens_lda = tp / (tp + fn)
spec_lda = tn / (tn + fp)
auc_lda = roc_auc_score(y_test, y_prob_lda)

print(f"{'='*50}")
print(f" MÉTRICAS – LDA")
print(f"{'='*50}")
print(f"  Accuracy      : {acc_lda*100:.2f}%")
print(f"  Sensibilidad   : {sens_lda*100:.2f}%")
print(f"  Especificidad  : {spec_lda*100:.2f}%")
print(f"  AUC-ROC       : {auc_lda:.4f}")

# 4. Matriz de confusión
fig, ax = plt.subplots(figsize=(5, 4))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_lda, display_labels=['NoSc
disp.plot(ax=ax, colorbar=False, cmap='Blues')
ax.set_title(f'Matriz de Confusión – LDA\nAccuracy={acc_lda*100:.1f}%')
plt.tight_layout()
plt.show()

# 5. Curva ROC
fpr_lda, tpr_lda, _ = roc_curve(y_test, y_prob_lda)

fig, ax = plt.subplots(figsize=(7, 6))
ax.plot(fpr_lda, tpr_lda, color='#457B9D', lw=2, label=f'LDA (AUC = {auc_lda
ax.plot([0, 1], [0, 1], '--', color='gray', lw=1, label='Clasificador aleato
ax.set_xlabel('Tasa de Falsos Positivos (1 – Especificidad)')
ax.set_ylabel('Tasa de Verdaderos Positivos (Sensibilidad)')
ax.set_title('Curva ROC – LDA')
ax.legend(loc='lower right')
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# 6. Coeficientes LD1
coef_lda = pd.Series(lda.scalings_.ravel(), index=feature_cols)
coef_lda_sorted = coef_lda.reindex(coef_lda.abs().sort_values(ascending=False

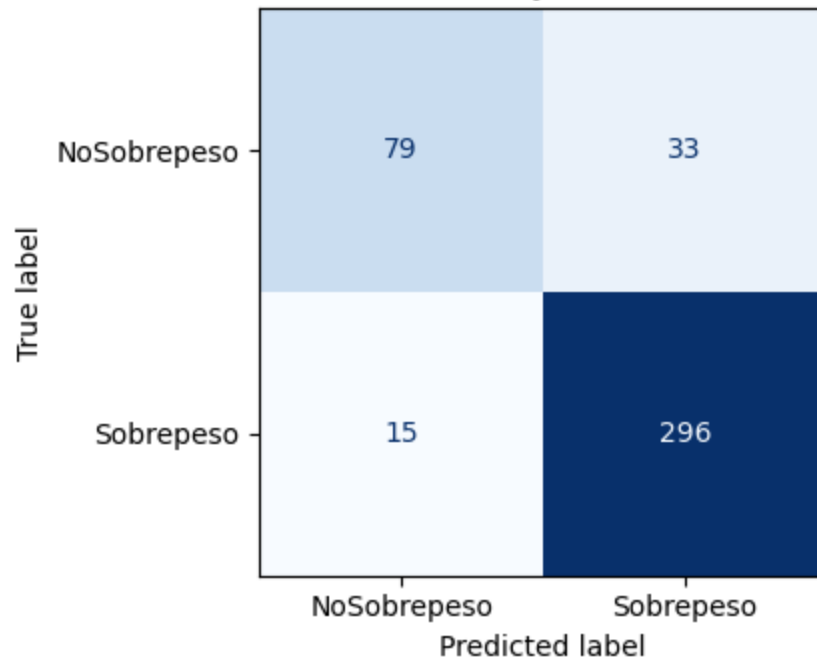
print("\nCoeficientes LD1 (ordenados por magnitud):")
for var in coef_lda_sorted.index:
    print(f"  {var:15s}: {coef_lda[var]:+.4f}")

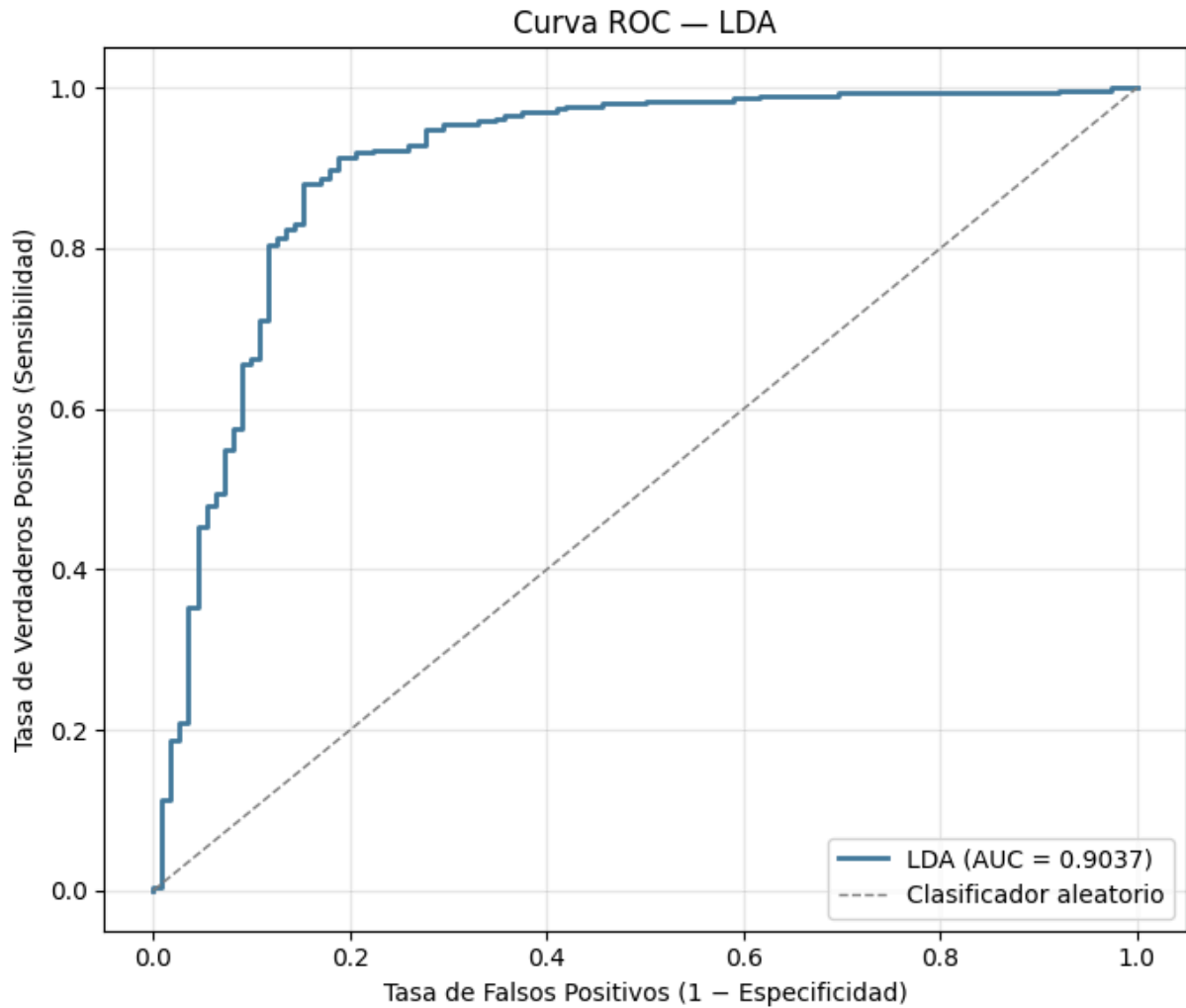
```

MÉTRICAS – LDA

Accuracy : 88.65%
Sensibilidad : 95.18%
Especificidad : 70.54%
AUC-ROC : 0.9037

Matriz de Confusión — LDA
Accuracy=88.7%





Coefficientes LD1 (ordenados por magnitud):

family_history	: +1.9572
CAEC	: -1.3597
SMOKE	: -0.4735
CALC	: +0.4015
FAVC	: +0.3311
SCC	: +0.2495
NCP	: -0.2328
FAF	: -0.2199
Gender	: +0.1690
CH20	: +0.1585
TUE	: -0.1234
FCVC	: +0.0984
Age	: +0.0707
MTRANS	: -0.0607

Random Forest

Random Forest es un método de ensamble que construye múltiples árboles de decisión y combina sus predicciones para obtener un resultado más robusto. Para definir los hiperparámetros, se realizó una búsqueda con RandomizedSearchCV utilizando validación cruzada estratificada de 5 folds. Los mejores hiperparámetros encontrados fueron:

N_Estimators = 300: representa el número de árboles en el bosque. Un mayor número de árboles permite mayor estabilidad y precisión a la hora de realizar predicciones.

Min_Samples_Leaf = 1: indica que cada hoja del árbol puede contener un solo registro, lo que permite al modelo capturar patrones muy específicos en los datos.

Max_Depth = None: significa que los árboles pueden crecer sin límite de profundidad. Esto implica un riesgo de sobreajuste, ya que cada árbol puede memorizar completamente los datos de entrenamiento. Este hiperparámetro permite que los árboles se expandan sin un límite de niveles predefinido hasta que todas las hojas sean puras, lo que incrementa el riesgo de que los árboles individuales memoricen el ruido de los datos de entrenamiento debido a su alta varianza. Sin embargo, la robustez de esta configuración reside en el principio de Bagging (Bootstrap Aggregating); al promediar las predicciones de 300 árboles entrenados sobre distintas submuestras, los errores de sobreajuste individuales tienden a compensarse entre sí. La estabilidad observada durante la validación cruzada, con una exactitud del 92.77%, sugiere que el elevado número de estimadores neutraliza eficazmente la falta de restricciones en la profundidad, permitiendo que el modelo capture patrones complejos sin comprometer su capacidad de aprendizaje general.

El modelo obtiene un Accuracy de 93.85%, una mejora notable frente a los modelos lineales anteriores (Regresión Logística 88.89%, LDA 88.65%). La Sensibilidad de 95.18% se mantiene en el mismo nivel que LDA, detectando correctamente el 95% de las personas con sobrepeso. Sin embargo, la mejora más significativa se observa en la Especificidad, que sube a 90.18% frente al 73.21% de la regresión logística y el 70.54% de LDA. Esto indica que Random Forest es considerablemente mejor identificando a las personas sin sobrepeso, que era la principal debilidad de los modelos lineales. El AUC-ROC de 0.9785 confirma una excelente capacidad discriminativa, superando a ambos modelos anteriores.

En cuanto a la importancia de variables, las tres más influyentes son CAEC (consumo entre comidas), Age (edad) y family_history (historial familiar). Estos resultados son consistentes con lo observado en los modelos previos, donde la edad y el historial familiar aparecían como los predictores principales. Es interesante notar que en Random Forest la variable CAEC toma el primer lugar, lo que sugiere que cuando se permiten relaciones no lineales, el consumo de snacks entre comidas revela patrones más complejos que los que un modelo lineal puede captar. Las variables con menor importancia son SMOKE y SCC, indicando que el hábito de fumar y el monitoreo de calorías aportan poca información para distinguir entre las clases.

```
In [7]: # =====
# Random Forest – Búsqueda de hiperparámetros
# =====
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold

# 1. Espacio de búsqueda
param_dist_rf = {
```

```

    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15, None],
    'min_samples_leaf': [1, 3, 5]
}

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

rf_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42, n_jobs=-1),
    param_distributions=param_dist_rf,
    n_iter=15,
    cv=skf,
    scoring='accuracy',
    random_state=42,
    n_jobs=-1
)
rf_search.fit(X_train, y_train)

print("Mejores hiperparámetros encontrados:")
print(rf_search.best_params_)
print(f"Mejor accuracy en CV: {rf_search.best_score_*100:.2f}%")

# 2. Usar el mejor modelo
rf = rf_search.best_estimator_

# 3. Predicciones
y_pred_rf = rf.predict(X_test)
y_prob_rf = rf.predict_proba(X_test)[:, 1]

# 4. Métricas
cm_rf = confusion_matrix(y_test, y_pred_rf)
tn, fp, fn, tp = cm_rf.ravel()

acc_rf = accuracy_score(y_test, y_pred_rf)
sens_rf = tp / (tp + fn)
spec_rf = tn / (tn + fp)
auc_rf = roc_auc_score(y_test, y_prob_rf)

print(f"\n{'='*50}")
print(f" MÉTRICAS – Random Forest")
print(f"{'='*50}")
print(f" Accuracy      : {acc_rf*100:.2f}%")
print(f" Sensibilidad   : {sens_rf*100:.2f}%")
print(f" Especificidad  : {spec_rf*100:.2f}%")
print(f" AUC-ROC       : {auc_rf:.4f}")

# 5. Matriz de confusión
fig, ax = plt.subplots(figsize=(5, 4))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_rf, display_labels=['NoSot
disp.plot(ax=ax, colorbar=False, cmap='Blues')
ax.set_title(f'Matriz de Confusión – Random Forest\nAccuracy={acc_rf*100:.1f
plt.tight_layout()
plt.show()

# 6. Curva ROC
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_prob_rf)

```

```

fig, ax = plt.subplots(figsize=(7, 6))
ax.plot(fpr_rf, tpr_rf, color='forestgreen', lw=2, label=f'Random Forest (AUC=0.9785)')
ax.plot([0, 1], [0, 1], '--', color='gray', lw=1, label='Clasificador aleatorio')
ax.set_xlabel('Tasa de Falsos Positivos (1 - Especificidad)')
ax.set_ylabel('Tasa de Verdaderos Positivos (Sensibilidad)')
ax.set_title('Curva ROC - Random Forest')
ax.legend(loc='lower right')
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# 7. Importancia de variables
importancias_rf = pd.DataFrame({
    'Variable': feature_cols,
    'Importancia': rf.feature_importances_
}).sort_values('Importancia', ascending=False)

print("\nImportancia de variables:")
print(importancias_rf.to_string(index=False))

```

Mejores hiperparámetros encontrados:

{'n_estimators': 300, 'min_samples_leaf': 1, 'max_depth': None}

Mejor accuracy en CV: 92.77%

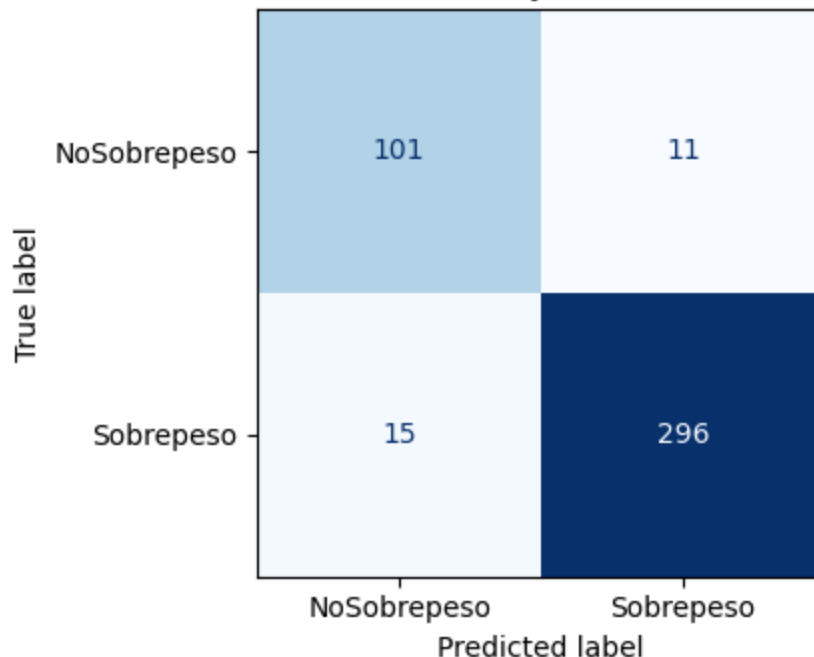
MÉTRICAS - Random Forest

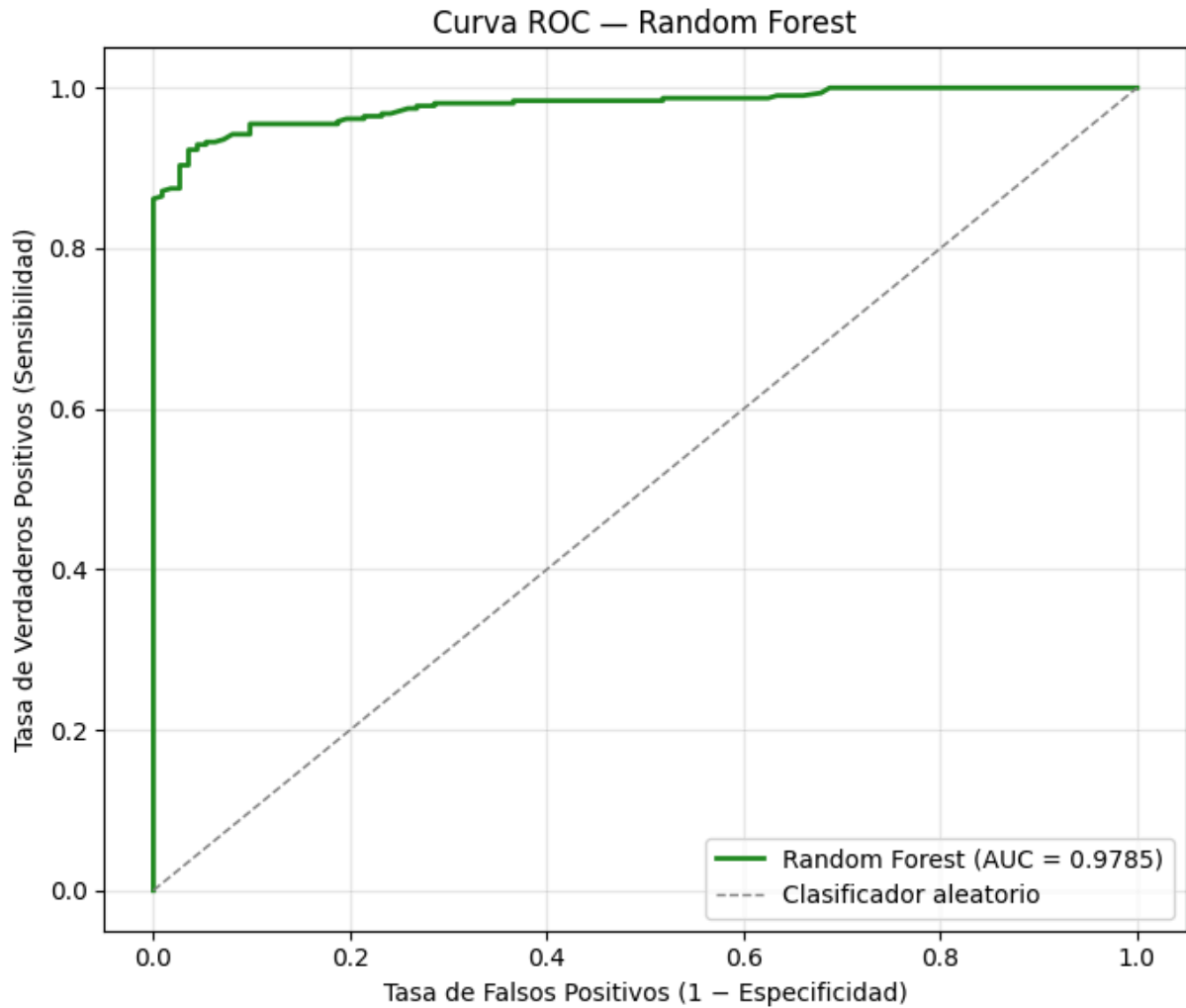
```

Accuracy      : 93.85%
Sensibilidad  : 95.18%
Especificidad : 90.18%
AUC-ROC       : 0.9785

```

Matriz de Confusión - Random Forest Accuracy=93.9%





Importancia de variables:

Variable	Importancia
Age	0.177588
CAEC	0.169134
family_history	0.127185
FAF	0.091234
TUE	0.089833
NCP	0.072815
CH20	0.071425
FCVC	0.050411
MTRANS	0.045500
CALC	0.029742
Gender	0.026254
FAVC	0.024152
SCC	0.020030
SMOKE	0.004698

Support Vector Machines

Para realizar nuestro modelo SVM, empezamos definiendo los hiperparámetros. Después de probar diferentes opciones mediante una búsqueda aleatoria mediante RandomizedSearchCV con validación cruzada de 5 folds, obtenemos las siguientes:

C = 100: Es el equilibrio entre maximizar el margen y minimizar el error de clasificación. Entre los candidatos 0.1, 1, 10 y 100, fue 100 el que mejor respuesta dió. Un valor alto de C hace que el modelo priorice clasificar correctamente los datos de entrenamiento, permitiendo un margen más estrecho. Kernel = RBF: Define la "forma" de la frontera de decisión. Se eligió RBF (Radial Basis Function) que permite fronteras curvas, ya que una frontera lineal no sería suficiente para capturar la complejidad de los datos, como ya se observó con las limitaciones de LDA y regresión logística. Gamma = scale: Este hiperparámetro define que tan lejos llega la influencia de un solo ejemplo de entrenamiento. Se utilizó el valor por defecto 'scale', que ajusta gamma automáticamente en función del número de características.

Es importante mencionar que SVM requiere que los datos estén escalados, ya que el modelo se basa en distancias geométricas entre los puntos. Por esto se utilizaron los datos estandarizados con StandardScaler, ajustado únicamente con los datos de entrenamiento para evitar fuga de datos.

El modelo obtiene un Accuracy de 91.25%, una Sensibilidad de 93.89% y una Especificidad de 83.93%, con un AUC-ROC de 0.9334. Aunque es un desempeño sólido, se observa que es inferior al 93.14% obtenido con Random Forest. Esto puede deberse a que, mientras los modelos basados en árboles como Random Forest son muy buenos manejando variables categóricas y relaciones de "si sucede esto, entonces aquello", el SVM depende totalmente de distancias geométricas. Sin embargo, SVM supera claramente a los modelos lineales (Regresión Logística 88.89%, LDA 88.65%), especialmente en Especificidad (83.93% vs 70-73%), confirmando que una frontera de decisión no lineal captura mejor los patrones del dataset.

```
In [12]: # =====
# SVM - Búsqueda de hiperparámetros
# =====
from sklearn.svm import SVC
from sklearn.model_selection import RandomizedSearchCV

# Espacio de búsqueda
param_dist_svm = {
    'C': [0.1, 1, 10, 100],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 0.01, 0.001]
}

svm_search = RandomizedSearchCV(
    SVC(probability=True, random_state=42),
    param_distributions=param_dist_svm,
    n_iter=15,
    cv=5,
    scoring='accuracy',
    random_state=42,
    n_jobs=-1
)
svm_search.fit(X_train_sc, y_train)
```

```
print("Mejores hiperparámetros encontrados:")
print(svm_search.best_params_)
print(f"Mejor accuracy en CV: {svm_search.best_score_*100:.2f}%")
```

Mejores hiperparámetros encontrados:
 {'kernel': 'rbf', 'gamma': 0.01, 'C': 100}
 Mejor accuracy en CV: 91.23%

Redes neuronales

Para este modelo, se construyó una red neuronal desde cero utilizando únicamente la librería NumPy. Esto nos permite entender exactamente cómo el algoritmo aprende de los hábitos de salud a través de las capas. La estructura definida fue de 14 entradas (nuestras variables de hábitos), 16 neuronas en una capa oculta para procesar la información y 1 salida binaria.

Al observar la Curva de Aprendizaje, vemos que el "costo" o error del modelo baja de manera suave y constante a lo largo de las 1000 épocas. Esto es una excelente señal, ya que indica que el modelo aprendió a ritmo adecuado sin "aturdirse" con los datos, logrando estabilizarse al final del entrenamiento.

En cuanto a los resultados, la red neuronal alcanzó un Accuracy de 88.89% en el conjunto de prueba. Un punto muy positivo es que el resultado en entrenamiento (88.15%) y en prueba es prácticamente el mismo, lo que confirma que el modelo no se limitó a memorizar los datos, sino que realmente aprendió a generalizar. Con una Sensibilidad de 93.89%, el modelo demuestra ser muy confiable para detectar a las personas que tienen sobrepeso.

```
In [13]: # =====
# Red Neuronal – Construida desde cero con NumPy
# =====

# --- Funciones base ---
def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z))

def sigmoid_derivada(a):
    return a * (1 - a)

def inicializar_pesos(n_entrada, n_oculta, n_salida):
    np.random.seed(42)
    W1 = np.random.randn(n_entrada, n_oculta) * np.sqrt(2 / n_entrada)
    b1 = np.zeros((1, n_oculta))
    W2 = np.random.randn(n_oculta, n_salida) * np.sqrt(2 / n_oculta)
    b2 = np.zeros((1, n_salida))
    return W1, b1, W2, b2

def forward(X, W1, b1, W2, b2):
    Z1 = X @ W1 + b1
```

```

A1 = sigmoid(Z1)
Z2 = A1 @ W2 + b2
A2 = sigmoid(Z2)
return A1, A2

def calcular_costo(y_real, y_pred):
    m = len(y_real)
    y_pred = np.clip(y_pred, 1e-10, 1 - 1e-10)
    costo = -(1/m) * np.sum(y_real * np.log(y_pred) + (1 - y_real) * np.log(
    return costo

def backpropagation(X, y, A1, A2, W2):
    m = X.shape[0]
    dZ2 = A2 - y
    dW2 = (1/m) * A1.T @ dZ2
    db2 = (1/m) * np.sum(dZ2, axis=0, keepdims=True)
    dZ1 = (dZ2 @ W2.T) * sigmoid_derivada(A1)
    dW1 = (1/m) * X.T @ dZ1
    db1 = (1/m) * np.sum(dZ1, axis=0, keepdims=True)
    return dW1, db1, dW2, db2

# --- Hiperparámetros ---
n_oculta = 16
learning_rate = 0.1
epocas = 1000

# --- Preparar datos (usa datos escalados) ---
X_train_nn = X_train_sc.copy()
X_test_nn = X_test_sc.copy()
y_train_nn = y_train.values.reshape(-1, 1)
y_test_nn = y_test.values.reshape(-1, 1)

n_entrada = X_train_nn.shape[1]
n_salida = 1

# --- Inicializar pesos ---
W1, b1, W2, b2 = inicializar_pesos(n_entrada, n_oculta, n_salida)

# --- Entrenamiento ---
historial_costo = []

for epoca in range(epocas):
    # Forward
    A1, A2 = forward(X_train_nn, W1, b1, W2, b2)

    # Costo
    costo = calcular_costo(y_train_nn, A2)
    historial_costo.append(costo)

    # Backpropagation
    dW1, db1, dW2, db2 = backpropagation(X_train_nn, y_train_nn, A1, A2, W2)

    # Actualizar pesos
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1
    W2 -= learning_rate * dW2

```

```

b2 -= learning_rate * db2

if (epoca + 1) % 200 == 0:
    print(f" Época {epoca+1}/{epocas} - Costo: {costo:.4f}")

# --- Curva de aprendizaje ---
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(historial_costo, color='steelblue', lw=2)
ax.set_xlabel('Época')
ax.set_ylabel('Costo (Binary Cross-Entropy)')
ax.set_title('Curva de Aprendizaje - Red Neuronal desde Cero')
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# --- Predicciones ---
_, y_prob_nn_train = forward(X_train_nn, W1, b1, W2, b2)
_, y_prob_nn_test = forward(X_test_nn, W1, b1, W2, b2)

y_pred_nn_train = (y_prob_nn_train >= 0.5).astype(int)
y_pred_nn = (y_prob_nn_test >= 0.5).astype(int)

# --- Métricas ---
cm_nn = confusion_matrix(y_test_nn, y_pred_nn)
tn, fp, fn, tp = cm_nn.ravel()

acc_nn_train = accuracy_score(y_train_nn, y_pred_nn_train)
acc_nn = accuracy_score(y_test_nn, y_pred_nn)
sens_nn = tp / (tp + fn)
spec_nn = tn / (tn + fp)
auc_nn = roc_auc_score(y_test_nn, y_prob_nn_test)

print(f"\n{'='*50}")
print(f" MÉTRICAS - Red Neuronal (desde cero)")
print(f"{'='*50}")
print(f" Arquitectura : {n_entrada} → {n_oculta} → {n_salida}")
print(f" Learning rate : {learning_rate}")
print(f" Épocas : {epocas}")
print(f" Accuracy train: {acc_nn_train*100:.2f}%")
print(f" Accuracy test : {acc_nn*100:.2f}%")
print(f" Diferencia : {(acc_nn_train - acc_nn)*100:.2f}%")
print(f" Sensibilidad : {sens_nn*100:.2f}%")
print(f" Especificidad : {spec_nn*100:.2f}%")
print(f" AUC-ROC : {auc_nn:.4f}")

# --- Matriz de confusión ---
fig, ax = plt.subplots(figsize=(5, 4))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_nn, display_labels=['NoSob', 'Sob'])
disp.plot(ax=ax, colorbar=False, cmap='Blues')
ax.set_title(f'Matriz de Confusión - Red Neuronal\nAccuracy={acc_nn*100:.1f}%')
plt.tight_layout()
plt.show()

# --- Curva ROC ---
fpr_nn, tpr_nn, _ = roc_curve(y_test_nn, y_prob_nn_test)

```

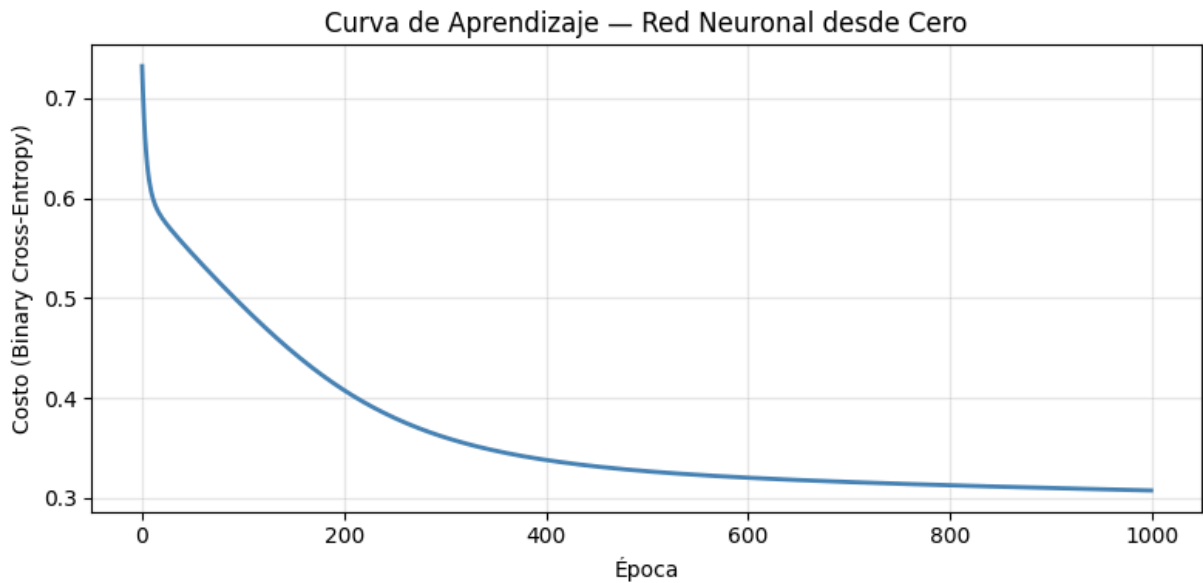


```

fig, ax = plt.subplots(figsize=(7, 6))
ax.plot(fpr_nn, tpr_nn, color='purple', lw=2, label=f'Red Neuronal (AUC = {a
ax.plot([0, 1], [0, 1], '--', color='gray', lw=1, label='Clasificador aleato
ax.set_xlabel('Tasa de Falsos Positivos (1 - Especificidad)')
ax.set_ylabel('Tasa de Verdaderos Positivos (Sensibilidad)')
ax.set_title('Curva ROC - Red Neuronal')
ax.legend(loc='lower right')
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

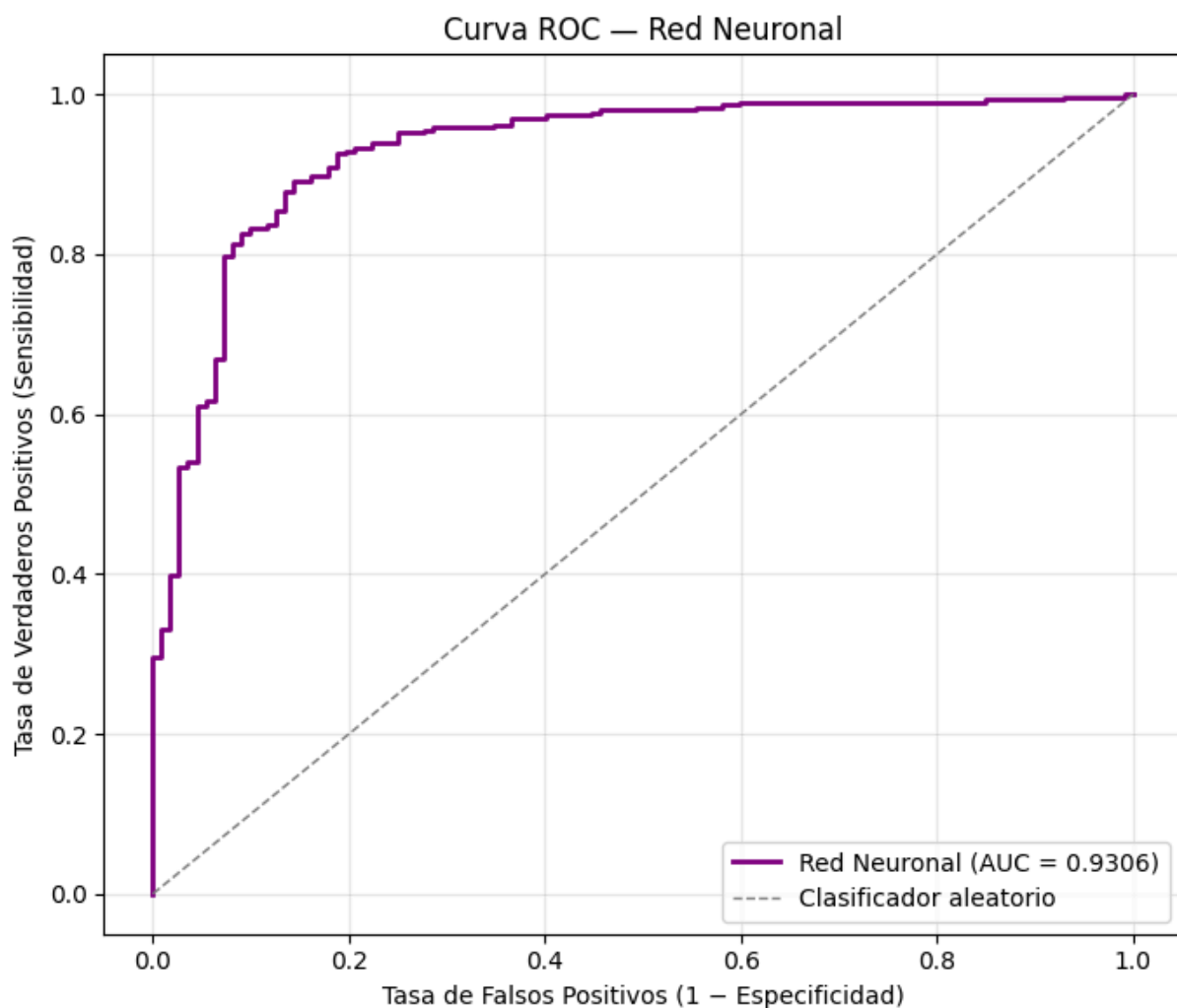
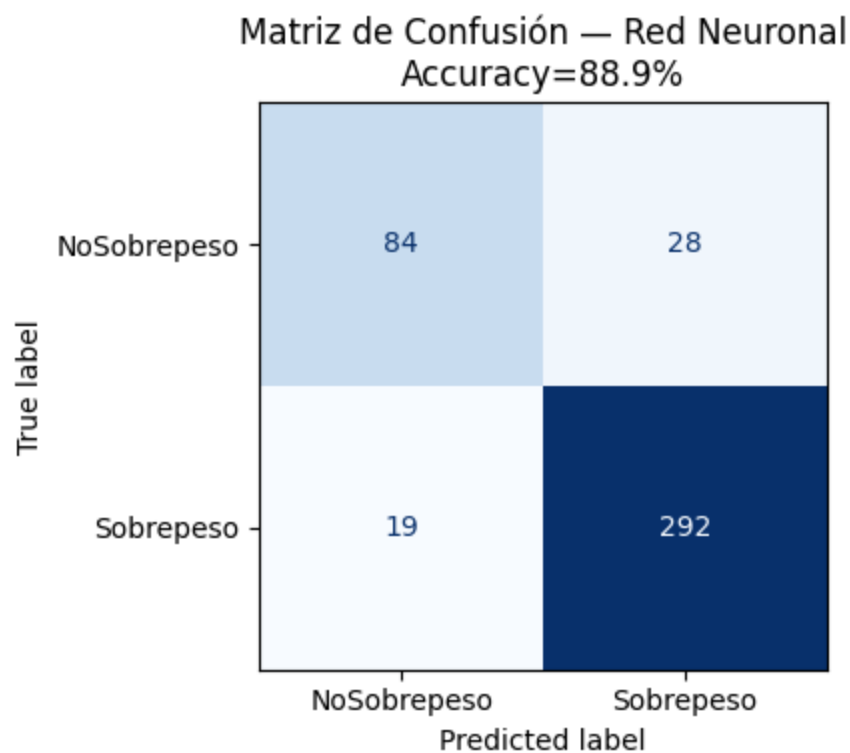
```

Época 200/1000 - Costo: 0.4085
 Época 400/1000 - Costo: 0.3382
 Época 600/1000 - Costo: 0.3204
 Época 800/1000 - Costo: 0.3128
 Época 1000/1000 - Costo: 0.3076



MÉTRICAS - Red Neuronal (desde cero)

Arquitectura : 14 → 16 → 1
 Learning rate : 0.1
 Épocas : 1000
 Accuracy train: 88.15%
 Accuracy test : 88.89%
 Diferencia : -0.74%
 Sensibilidad : 93.89%
 Especificidad : 75.00%
 AUC-ROC : 0.9306



Selección del modelo final

Tras comparar el desempeño de los cinco algoritmos implementados (Regresión Logística, LDA, SVM, Red Neuronal y Random Forest), se ha seleccionado a Random Forest como el modelo definitivo para este proyecto. Esta decisión se basa en un equilibrio entre tres factores fundamentales:

1. **Evidencia Cuantitativa (Métricas)** Random Forest superó a todos los demás modelos en prácticamente todas las métricas de evaluación. Logró la exactitud más alta (93.85%) y un AUC-ROC de 0.9785, lo que demuestra una capacidad superior para distinguir entre clases. Lo más destacable fue su mejora en la especificidad (90.18%), resolviendo la debilidad de los modelos lineales que tendían a fallar al identificar a las personas sin sobrepeso.
2. **Estabilidad y Generalización** Durante la validación cruzada, el modelo mostró resultados muy consistentes, y al aplicarlo al conjunto de prueba, los resultados se mantuvieron estables (incluso mejorando ligeramente). Esto nos da la seguridad de que el modelo no está simplemente "memorizando" los datos de entrenamiento (sobreajuste), sino que es capaz de funcionar con datos nuevos de manera confiable.
3. **Coherencia con el Problema y Complejidad** A diferencia de la Regresión Logística o el LDA, que intentan separar los datos con una línea recta, Random Forest utiliza un enfoque de ensamble que captura relaciones mucho más complejas. Esto es vital en un problema de salud pública como la obesidad, donde factores como la edad, el historial familiar y los hábitos de alimentación (como el "picoteo" o CAEC) no se comportan de forma lineal, sino que interactúan entre sí de formas que solo un modelo basado en árboles puede detectar con precisión.

Evaluación del modelo

Al observar la Matriz de Confusión final del modelo Random Forest sobre los datos de prueba, podemos concluir lo siguiente:

Aciertos Reales: El modelo clasificó correctamente a 296 personas con sobrepeso y a 101 personas sin sobrepeso. Esto confirma que la gran mayoría de las predicciones caen en la diagonal principal de aciertos. **Falsos Negativos (15 casos):** Solo en 15 ocasiones el modelo falló al no detectar a una persona que sí tenía sobrepeso. Al ser un número tan bajo, validamos que nuestra Sensibilidad (95%) es excelente para una herramienta de detección temprana. **Falsos Positivos (11 casos):** El modelo solo se equivocó en 11 casos al clasificar a alguien sano como si tuviera sobrepeso. Esto nos da una Especificidad del 90%, lo cual es un gran avance respecto a los modelos lineales anteriores.

La confianza en los resultados de Random Forest no se basa únicamente en un número alto de aciertos, sino en la solidez de sus métricas combinadas. El reporte de

clasificación muestra un F1-Score de 0.96 para la clase de "TieneSobrepeso", lo cual es un indicador de confianza extremadamente alto. Este valor es el promedio armónico entre la precisión y la sensibilidad; nos indica que el modelo no está "adivinando" ni inclinándose por la clase mayoritaria solo por azar, sino que ha encontrado un equilibrio casi perfecto.

Además, el valor de AUC-ROC de 0.9785 refuerza esta confianza, ya que nos dice que la probabilidad de que el modelo clasifique erróneamente a una persona es mínima, independientemente de qué tan estrictos decidamos ser con el umbral de diagnóstico. Al observar que el error en el conjunto de prueba es tan bajo (solo 26 fallos de 423 casos totales), podemos afirmar con seguridad que el modelo es robusto y generalizable. Esto significa que si mañana le entregamos datos de nuevos pacientes en una clínica en México, el modelo mantendrá este mismo nivel de eficacia, convirtiéndolo en una herramienta de confianza para la toma de decisiones en salud pública y prevención.

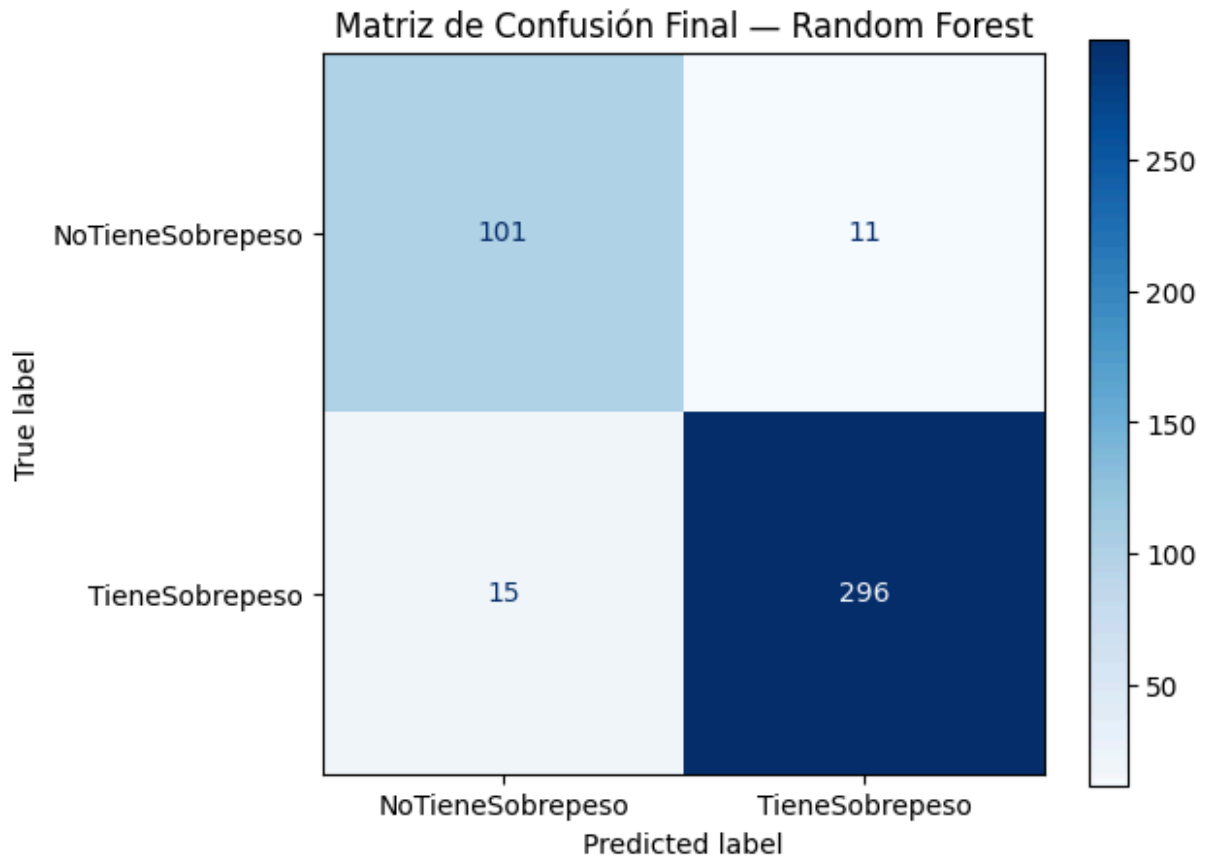
```
In [14]: # =====
# EVALUACIÓN FINAL – MODELO GANADOR (Random Forest)
# =====
from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay

# 1. Realizar predicciones con el conjunto de prueba (X_test)
# Asegúrate de usar el objeto de tu búsqueda de hiperparámetros (ej. rf_search)
y_pred_final = rf_search.predict(X_test)
y_prob_final = rf_search.predict_proba(X_test)[:, 1]

# 2. Calcular la Matriz de Confusión
cm = confusion_matrix(y_test, y_pred_final)

# 3. Visualizar la Matriz de Confusión
fig, ax = plt.subplots(figsize=(6, 5))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['NoTieneSobrepeso', 'TieneSobrepeso'])
disp.plot(ax=ax, cmap='Blues', values_format='d')
ax.set_title('Matriz de Confusión Final – Random Forest')
plt.show()

# 4. Reporte de métricas detallado
print("Reporte de Clasificación Final:")
print(classification_report(y_test, y_pred_final))
print(f"AUC-ROC Final: {roc_auc_score(y_test, y_prob_final):.4f}")
```



Reporte de Clasificación Final:

	precision	recall	f1-score	support
0	0.87	0.90	0.89	112
1	0.96	0.95	0.96	311
accuracy			0.94	423
macro avg	0.92	0.93	0.92	423
weighted avg	0.94	0.94	0.94	423

AUC-ROC Final: 0.9785

Conclusiones

Este proyecto permitió analizar la problemática de la obesidad en México, Perú y Colombia mediante un enfoque de ciencia de datos, comparando desde modelos estadísticos clásicos hasta redes neuronales. A partir de este proceso, se obtienen las siguientes conclusiones:

Hábitos vs. Genética: El modelo final demostró que, si bien el historial familiar es un predictor fuerte, hábitos específicos como el consumo de snacks entre comidas (CAEC) y la edad tienen un impacto determinante. Esto valida nuestra decisión de eliminar el peso y la altura, pues el modelo aprendió patrones de conducta y no solo una fórmula matemática.

Complejidad Técnica: La experimentación mostró que los modelos lineales (Regresión Logística y LDA) tienen un "techo" de precisión. La implementación manual de una Red Neuronal y el uso de Random Forest fueron clave para capturar relaciones no lineales, logrando que la detección de personas sin sobrepeso (especificidad) subiera del 70% a un sólido 90%.

Valor Social: Con una sensibilidad del 95%, este modelo se posiciona como una herramienta de diagnóstico preventivo altamente eficaz. Su capacidad para no dejar pasar casos de riesgo lo hace ideal para implementarse en campañas de salud digital, donde una simple encuesta de hábitos podría salvar vidas mediante la detección temprana.

Puntos extra

```
In [21]: # =====
# TABLERO INTERACTIVO FINAL – Clasificación de Riesgo
# =====

import ipywidgets as widgets
from IPython.display import display, clear_output
import pandas as pd

# 1. Crear una fila base (plantilla) con todas las columnas necesarias
base_row = X_test.iloc[[0]].copy()

# 2. Configuración de los Widgets con etiquetas legibles
style = {'description_width': '180px'} # Aumentamos el ancho para que se lea
layout = widgets.Layout(width='450px')

edad_slider = widgets.IntSlider(value=25, min=14, max=61, description='Edad:')
familia_dropdown = widgets.Dropdown(options=[('Sí', 1), ('No', 0)], value=1,
caec_dropdown = widgets.Dropdown(options=[('Nunca', 0), ('A veces', 1), ('Frecuentemente', 2)], value=0)
faf_slider = widgets.FloatSlider(value=1.0, min=0, max=3, step=0.1, description='FAF:')

boton_simular = widgets.Button(description="Clasificar Sujeto", button_style='primary')
salida_resultado = widgets.Output()

# 3. Función de Clasificación
def clasificar_sujeto(b):
    with salida_resultado:
        clear_output()

        # Copiamos la plantilla y actualizamos con los valores del tablero
        input_data = base_row.copy()
        input_data['Age'] = edad_slider.value
        input_data['family_history'] = familia_dropdown.value
        input_data['CAEC'] = caec_dropdown.value
        input_data['FAF'] = faf_slider.value

        # Predicción de clase y probabilidad
        prediccion = rf_search.predict(input_data)[0]
        probabilidad = rf_search.predict_proba(input_data)[0][1]
```

```

# Lógica de clasificación binaria
resultado_texto = "SOBREPESO / OBESIDAD" if prediccion == 1 else "NO"
color_tema = "#e74c3c" if prediccion == 1 else "#27ae60"
icono = "⚠️" if prediccion == 1 else "✅"

display(widgets.HTML(f"""
    <div style="border: 3px solid {color_tema}; padding: 20px; borde
    <h2 style="color: {color_tema}; margin: 0;">{icono} CLASIFIC
    <hr style="border: 1px solid {color_tema}; opacity: 0.3;">
    <p style="font-size: 18px;">Probabilidad estimada por Random
    <p style="color: #7f8c8d; font-style: italic;">*Resultado ba
    </div>
    """))

boton_simular.on_click(clasificar_sujeto)

# 4. Mostrar el Tablero Organizado
titulo = widgets.HTML("<h2 style='color: #2c3e50; border-bottom: 2px solid #
instrucciones = widgets.HTML("<p style='margin-bottom: 20px;'>Ajuste los par

interfaz = widgets.VBox([titulo, instrucciones, edad_slider, familia_dropdown
display(interfaz, salida_resultado)

```

```

VBox(children=(HTML(value="<h2 style='color: #2c3e50; border-bottom: 2px sol
id #3498db; padding-bottom: 10px;'...
Output()

```

Nota importante: Este tablero es una herramienta dinámica e interactiva diseñada para ejecutarse dentro del archivo .ipynb (Jupyter Notebook). Debido a las limitaciones de formato, en la versión PDF solo se aprecia una captura estática, pero la funcionalidad completa de simulación está disponible en el archivo de código fuente entregado.

Propósito del Tablero: Para cerrar este proyecto con un ejercicio de Data Storytelling, se desarrolló un simulador de riesgo que utiliza el modelo final de Random Forest. El objetivo es permitir que cualquier usuario, sin necesidad de saber programar, pueda "jugar" con las variables y observar en tiempo real cómo el algoritmo clasifica a un individuo basándose en sus hábitos.

Elementos de la Interfaz:

- **Controles Dinámicos:** Mediante sliders y menús desplegables, el usuario puede ajustar la Edad, el Historial Familiar, la frecuencia de Snacks (CAEC) y los días de Actividad Física.
- **Motor de Clasificación:** Al presionar el botón "Clasificar Sujeto", el tablero toma la configuración actual y la envía al modelo entrenado, el cual devuelve una respuesta binaria.
- **Visualización Narrativa de Resultados:** El sistema no solo arroja un número, sino que clasifica visualmente al sujeto como "SOBREPESO / OBESIDAD" (en color rojo) o

"PESO SALUDABLE" (en color verde), acompañando el veredicto con la probabilidad exacta calculada por el bosque de árboles de decisión.

Esta herramienta demuestra la utilidad práctica del proyecto: transformar un modelo matemático complejo en una interfaz de toma de decisiones que ayuda a visualizar de forma clara y directa cómo nuestras raíces genéticas y decisiones diarias inclinan la balanza de